



AI TEAM TRAINING '27

TASK 1 PHASE I

World Cup Theme

Welcome to the MIA World Cup Broadcast & Analytics Center! You're the newest artificial intelligence engineers on the crew, and your skills are needed to keep the tournament running on screen, on air, and at the gates. Today, you'll be working on three critical systems built for the World Cup Final week.

Task 1.1: Group Stage Standings Engine

The Scenario:

It's the group stage, and the broadcast graphics team needs the live standings table on screen after every match Played, Won, Drawn, Lost, Goals For, Goals Against, Goal Difference, and Points, sorted the same way FIFA sorts it. Your job is to build the engine that computes that table.

Your Objective:

Write an interactive Python program that asks the user for the score of each match in a 4-team group (6 matches total, since every team plays every other team once), then computes and prints the final standings table, sorted correctly.

How the data is organized:

Think of the standings as one dictionary where each team name maps to its own dictionary of stats, all starting at zero:

```
standings = { "ARG": {"P": 0, "W": 0, "D": 0, "L": 0, "GF": 0, "GA": 0, "GD": 0, "Pts": 0},
               "MEX": {"P": 0, "W": 0, "D": 0, "L": 0, "GF": 0, "GA": 0, "GD": 0, "Pts": 0},
               "POL": { ... same shape ... }, "KSA": { ... same shape ... },
               }
```

Walking through one match:

Say the user enters ARG beat MEX 2–0. Here's exactly what should change:

Before:

ARG: P=0 W=0 D=0 L=0 GF=0 GA=0 GD=0 Pts=0

MEX: P=0 W=0 D=0 L=0 GF=0 GA=0 GD=0 Pts=0

After process_match(standings, "ARG", "MEX", 2, 0):

ARG: P=1 W=1 D=0 L=0 GF=2 GA=0 GD=+2 Pts=3 MEX: P=1 W=0 D=0 L=1 GF=0
GA=2 GD=-2 Pts=0

Both teams get +1 Played. ARG scored 2 and conceded 0, so their GF goes up by 2 and GA stays put; MEX is the reverse. Since ARG scored more, they get the win (+3 points, +1 W); MEX gets the loss (+1 L, +0 points). If the score had been equal, both teams would get +1 D and +1 point instead. GD is always just GF minus GA, recalculated after every update.

Technical Requirements:

1. Represent the standings as a dictionary keyed by team name, where each value is itself a dictionary holding that team's P, W, D, L, GF, GA, GD, and Pts, all starting at 0.
2. Create a function `process_match(standings, team1, team2, team1_goals, team2_goals)` that updates both teams' stats for one match: Played goes up by 1 for both, goals scored/conceded are added to GF/GA, the winner gets 3 points (or both teams get 1 point each on a draw), and GD is recalculated as GF minus GA.
3. The four team names and the six matchups (ARG vs MEX, ARG vs POL, ARG vs KSA, MEX vs POL, MEX vs KSA, POL vs KSA) are fixed in your script. For each matchup, prompt the user for that match's score in the format "X-Y" (e.g. "2-0"), split it into two integers, and pass them into `process_match`. Do not hardcode the scores themselves those come from the user.
4. Create a function `print_standings(standings)` that prints every team's row in neatly aligned columns, sorted using the tie-break order below.
5. When printing Goal Difference specifically, show a leading '+' for positive values (e.g. +3), a leading '-' for negative values (e.g. -4), and no sign at all for exactly 0 (just 0, not +0). This formatting is required, not just cosmetic see the example session below for exactly what's expected.
6. After all 6 scores have been entered, call `print_standings` once to show the final table.
7. Your sort must actually implement all three tie-break levels below sorting by points alone is not enough.

Sorting the table:

Real World Cup standings are sorted in this order: first by Points (most first); if two or more teams are tied on points, by Goal Difference (best first); if they're still tied, by Goals Scored (most first).

Example Session 1:

Enter score for ARG vs MEX (format: 2-0): 2-0
Enter score for ARG vs POL (format: 2-0): 1-1
Enter score for ARG vs KSA (format: 2-0): 2-1
Enter score for MEX vs POL (format: 2-0): 1-1
Enter score for MEX vs KSA (format: 2-0): 2-1
Enter score for POL vs KSA (format: 2-0): 2-0

Team	P	W	D	L	GF	GA	GD	Pts
ARG	3	2	1	0	5	2	+3	7
POL	3	1	2	0	4	2	+2	5
MEX	3	1	1	1	3	4	-1	4
KSA	3	0	0	3	2	6	-4	0

Bonus Section (for Data-Driven Analysts):

- **Input Validation:** Since a real person is typing scores in live, someone will eventually mistype (e.g. "two-zero" instead of "2-0", or leaving it blank). If the input isn't in the expected "X-Y" numeric format, re-prompt instead of crashing.
- **Tie-Break Test:** Enter scores where two teams end up level on both points and goal difference, so your sort has to fall back to Goals Scored to separate them. If your table doesn't order them correctly, your sort isn't actually implementing all three levels.
- **Head-to-Head Rule (Advanced):** FIFA's real tie-break actually checks the head-to-head result between two teams before falling back to goals scored, but only when exactly two teams are tied. Try adding that as an extra tie-break level.

Task 1.2: Stadium Gate Ticket Codec

The Scenario:

Stadium gates scan a barcode on every ticket before letting fans in. To catch forged or tampered tickets instantly, without needing a live database lookup at every gate, each barcode needs to carry its own built-in tamper check — a checksum baked into the code itself.

A checksum is a small value calculated from a piece of data that acts like a fingerprint for it: if even one character of the original data changes, the checksum comes out different. Comparing checksums instantly tells you whether the data is still intact or was tampered with, without re-checking the whole thing.

Your Objective:

Implement a TicketCodec class with two methods:

1. **encode(self, ticket_id):** Takes a ticket ID string and returns a barcode string that embeds a checksum you design.
2. **decode(self, barcode):** Takes a barcode string, recomputes the checksum from the ticket portion, and compares it to the checksum embedded in the barcode. Returns the original ticket ID if it matches, or flags the ticket as corrupted if it doesn't.

Technical Requirements:

1. Design your own checksum method, there's no single required formula, but you must be able to explain how it works in a comment.
2. Your checksum must be non-trivial: any single-character change to the ticket ID must change the computed checksum. A method that ignores part of the content, or that can't tell a valid ticket from a tampered one, does not satisfy this requirement, a corrupted ticket must never be able to decode as valid.
3. Your solution must work for any reasonable ticket ID (letters and numbers).
4. Your script should demonstrate that your code works by:
 - Encoding 2–3 sample ticket IDs and printing the resulting barcodes.
 - Decoding each barcode as-is and showing it succeeds.
 - Hand-corrupting one character in a barcode and decoding it again, showing it gets flagged.

Example (illustrative — your checksum values will differ):

`encode("MIA2026GATE7") -> "MIA2026GATE7-XX"`

`decode("MIA2026GATE7-XX") -> "MIA2026GATE7"` (valid)

`decode("MIA2026GATE4-XX") -> "CORRUPTED TICKET"` (tampered)

"XX" stands in for whatever checksum your own algorithm produces — it will be different for every student's solution, and that's expected

Task 1.3: FIFA World Cup Simulation System

Object-Oriented Programming (OOP) Blueprint & Exact Engine Logic

This technical document outlines the comprehensive Object-Oriented Programming (OOP) structural architecture and explicit execution logic for a professional-grade FIFA World Cup Match Simulation Engine. Built to completely eliminate behavioral uncertainty, this system utilizes strict mathematical formulas and deterministic state machines to accurately model a single realistic match, including squad dynamics and stamina depletion

1. Core Class Architecture

- **Class: Player** Represents an individual athlete. Tracks concrete performance metrics (Attacking Skill, Defensive Skill, Stamina) and models granular degradation over elapsed playing time.
- **Class: Team** Acts as an aggregate collection of Player instances. Manages active 11-player lineups, field position weighting, bench availability, and calculates real-time aggregated tactical matrices.
- **Class: MatchEvent** An immutable structural record logging definitive occurrences on the pitch. Each instance captures an exact timestamp, precise category flags, targeted players, and strict outcome text.
- **Class: Match** The central procedural state engine. Controls match phases (Regulation, Finished), tracks current scoring thresholds, runs the minute-by-minute simulation loop, and drives rule execution.

2. Explicit Technical Blueprint

Class: Player

Attributes & State Variables

Attribute	Type	Description
name	String	The unique identity string of the athlete.
position	Enum	Restricted to [FORWARD, MIDFIELDER, DEFENDER, GOALKEEPER].
base_attack	Integer	Fixed raw offensive metric scaled strictly from 1 to 100.
base_defense	Integer	Fixed raw defensive metric scaled strictly from 1 to 100.
stamina	Float	Continuous dynamic parameter initialized to 100.0.

Internal Logic & Methods

- **deplete_stamina(rate)**: Deducts the designated rate from the stamina attribute every simulated block. Stamina floor is capped firmly at 10.0.
- **get_effective_attack()**: Returns $(\text{base_attack} * (\text{stamina} / 100.0))$.
- **get_effective_defense()**: Returns $(\text{base_defense} * (\text{stamina} / 100.0))$.

Class: Team

Attributes & State Variables

Attribute	Type	Description
country_name	String	The unique international name of the squad.
roster	List<Player>	Full container of 26 registered squad players.
active_lineup	List<Player>	Exactly 11 active Player objects mapped onto the field.
bench	List<Player>	Derived, not separately stored: roster minus active_lineup = 15 players. This is the exact pool execute_substitution() checks player_in against.
substitutions_remaining	Integer	Initialized to 5 for the match.

Internal Logic & Methods

- **get_aggregate_attack()**: Iterates through the active_lineup, sums the output of
- **get_effective_attack()** for FORWARD and MIDFIELDER positions, and divides by the count of those active positions.
- **get_aggregate_defense()**: Iterates through the active_lineup, sums the output of
- **get_effective_defense()** for DEFENDER and GOALKEEPER positions, and divides by the count of those active positions.
- **execute_substitution(player_out, player_in)**: Verifies substitutions_remaining > 0, player_out is in active_lineup, and player_in is on the bench. Swaps their locations, decrements substitutions_remaining by 1.

Class: MatchEvent

Attributes & State Variables

Attribute	Type	Description
event_id	String	Unique identifier assigned at creation; never reused.
event_type	Enum	Restricted to [GOAL, SUBSTITUTION, HALF_TIME, FULL_TIME].
minute	Integer	The exact simulated minute the event occurred.
team	Team Instance	The team associated with the event.
player	Player Instance	The specific player involved; null for team-level events like HALF_TIME.
outcome_text	String	Fixed, human-readable description of exactly what happened.

As an immutable record, MatchEvent exposes no setters — every field above is written once, inside the constructor, and can never be altered afterward. This is what guarantees Match.timeline stays a tamper-proof, trustworthy history of the game.

Internal Logic & Methods

- **to_string():** The only method exposed. Formats event_type, minute, team, and player into the standard timeline display string — read-only, never mutates state.

Class: Match

Attributes & State Variables

Attribute	Type	Description
home_team	Team	Assigned host/first squad object.
away_team	Team	Assigned visitor/second squad object.
home_score	Integer	Running tally of goals scored by home team.
away_score	Integer	Running tally of goals scored by away team.
current_minute	Integer	The absolute evaluation index from 1 to 90.
timeline	List<MatchEvent>	Chronological historical list of all evaluated match milestones.
phase	Enum	State engine indicator constrained to [REGULATION, FINISHED].

Internal Logic & Methods

- **run_minute_tick():** Increases current_minute by 1. Applies stamina decay to all active players (see the Stamina Degradation formula in Section 3). Then calls process_goal_attempt() for both teams to evaluate that minute's event thresholds.
- **process_goal_attempt(attacking_team, defending_team):** Samples an attack rating from attacking_team.get_aggregate_attack() and compares it against defending_team.get_aggregate_defense(). If attack wins, increments score and appends a **GOAL** MatchEvent.

3. Mathematical Simulation Logic Engine

To remove all visual ambiguity, the game engine resolves actions through exact mathematical inequalities rather than general descriptions. Every discrete minute, a standard probability event matrix is queried.

- **Stamina Degradation — How It Works**

Every active player loses stamina once per simulated minute, via `run_minute_tick()`. The loss is a fixed rate, and it can never push stamina below the hard floor of 10.0 (a player is always able to keep playing, just at reduced effectiveness).

- **Base_Decay = 0.5:** stamina points per minute, applied every minute of the match. This value is subtracted from the player's current stamina each minute, then clamped at the 10.0 floor.

- **Worked example:** A player enters minute 60 with 45.0 stamina. Effective loss = 0.5/min, so at minute 65 their stamina is $45.0 - (0.5 \times 5) = 42.5$. This continues until either the match ends at minute 90 or the 10.0 floor is reached.

- **Full Simulation Formula Matrix:**

Simulation Check	Governing Mathematical Formula
Stamina Degradation	$\text{Stamina}(t) = \text{Max}(10.0, \text{Stamina}(t-1) - \text{Base_Decay})$
Attempt Frequency	Each minute, each team has a 10% chance of generating a genuine scoring attempt: <code>random.random() < 0.10</code> . No attempt that minute means no goal check runs at all.
Goal Event Determination	If an attempt occurs: Goal scored if $(\text{Aggregate_Attack} * \text{Random}(0.75, 1.25)) > (\text{Aggregate_Defense} * 1.3 * \text{Random}(0.80, 1.20))$

Why the extra Attempt Frequency step and the 1.3x defense margin: checking every single minute with no throttling and no margin produces wildly unrealistic scorelines (both teams scoring 40+ times in one match). Gating attempts to a 10% per-minute chance and requiring attack to clearly outweigh a strengthened defense keeps results in a believable range (roughly 1–3 goals per team) while still rewarding the stronger squad.

4. Definitive Phase Transition Workflow

The match progression logic follows a simple two-state trajectory:

- ⚡ **Regulation Phase:** Minutes 1 to 90 are fully simulated.
- ⚡ **Match Result:** At minute 90, if a score disparity exists, phase transitions to **FINISHED** and the team with the higher score wins. If the score is tied at minute 90, phase transitions to **FINISHED** and the match ends in a **DRAW**.

Bonus Section (for Championship-Level Engineers):

- Replace the simple tiebreaker with a real third Phase — Penalties — and reuse the kick-by-kick board style from Task 1.1 to display it.
- Give each team 1–2 bench players and a substitution method that automatically swaps out whichever active player currently has the lowest stamina.
- Add a basic discipline system: a player who racks up enough incidents gets pulled from the active lineup — and make sure your Team's aggregate calculations don't break if a position group ends up empty.
- Widen the Goal Determination formula with a random swing on each side (e.g. multiply Aggregate_Attack by a random factor between 0.85 and 1.15 before comparing) so evenly matched teams don't always land near a flat 50/50 — this mirrors how real form/luck varies possession to possession.

BONUS: AI Model – Controlled Match Engine

As an optional enhancement, the deterministic rule engine above can be paired with a trained AI model that makes the tactical decisions a human coach would normally make — formation, substitution timing, and risk posture — while the underlying Match and Team classes keep resolving the actual on-pitch math untouched. This keeps results fair and reproducible while adding realistic, adaptive coaching behavior.

Class: MatchAI (extends coaching logic onto Team)

Attributes & State Variables

Attribute	Type	Description
<code>model</code>	AIModel	Reference to the trained decision-making model (e.g. a policy network or LLM-based agent).
<code>controlled_team</code>	Team	The Team instance this AI is responsible for coaching.
<code>risk_tolerance</code>	Float	0.0–1.0 dial the model can adjust mid-match to play safer or more aggressively.
<code>decision_log</code>	List	Human-readable record of every tactical decision the model made and why.

Internal Logic & Methods

- **observe_state(match)**: Serializes the current score, minute, phase, and stamina levels into a state vector the model can read.
- **decide_action()**: Feeds the observed state into the model and returns one of **SUBSTITUTE**, **CHANGE_FORMATION**, **HOLD**, or **PUSH_ATTACK**.
- **apply_decision(action)**: Executes the chosen action against controlled_team e.g. calling **execute_substitution()** and appends the reasoning to decision_log.

Hint: implement `decide_action()` by calling a real LLM API — e.g. Google Gemini — with the state from `observe_state()`, and parsing its reply into one of the four actions above.

What Each Decision Actually Changes

- **SUBSTITUTE:** picks the `active_lineup` player with the lowest current stamina as `player_out`, and the highest-rated bench player at that same position as `player_in`, then calls `execute_substitution(player_out, player_in)` outright — a fatigue-driven swap that keeps the formation intact and simply calls existing Team logic, no new math involved.
- **CHANGE_FORMATION:** doesn't add or remove players. It re-tags the same 11 `active_lineup` members across the position buckets that `get_aggregate_attack()/get_aggregate_defense()` already read from, shifting both the numerator and denominator of each average:

Formation	Defense Bucket	Attack Bucket	Net Tactical Effect
Defensive (5-3-2)	6 players	5 players	Aggregate_Defense is averaged over more players – steadier, lower-variance defense
Balanced (4-4-2)	5 players	6 players	Even split – the baseline formation assumed everywhere else in this document
Attacking (3-4-3)	4 players	7 players	Aggregate_Attack is averaged over more players – higher attack output, thinner defense

HOLD and PUSH_ATTACK — Live Risk Dial

HOLD and PUSH_ATTACK don't call a Team method or touch the position buckets at all — instead they nudge `risk_tolerance` up or down by 0.2 (clamped to [0.0, 1.0]), and that value feeds back into the exact Section 3 formulas as a live multiplier:

Simulation Check	Multiplier Applied At	<code>risk_tolerance = 1.0</code>	<code>risk_tolerance = 0.0</code>
Attack	upper bound $1.25 + (\text{risk_tolerance} - 0.5) \times 0.4$	1.45 – bigger swings	1.05 – safer attack
Stamina decay	$(\text{Base_Decay}) \times (1 + (\text{risk_tolerance} - 0.5) \times 0.4)$	1.2x – extra fatigue	0.8x – energy conserved

Integration point:

call `observe_state()` -> `decide_action()` -> `apply_decision()` once per `run_minute_tick()`, so coaching stays as responsive as the simulation clock itself.

Submission Guidelines

- You will submit one zip file including your proposed solution to each one of the tasks in its own environment (directory/python files).
- Name each file with its task number.
- For illustration questions, you could either write a md file, or any readable text format with the task number.
- You can submit your code multiple times, only the last submission will be taken into consideration.
- The Task's deadline is **23/7 11:59 PM**.
- It's highly encouraged to do the bonus tasks - Including functions, comments, and making your code clean, well-documented and readable will give bonus points.
- We are here to help!! It is highly discouraged to cheat, the goal of the process is learning, so keep an eye for that.